ООД

Что конкретно изменено в каждом файле

`include/memento/CanvasMemento.h` — новый файл

Что добавлено:

- Класс `CanvasMemento` — снимок всего холста.
- Поле `std::vector<std::unique_ptr<IShape>> m_shapes` — клоны всех фигур.
- Конструктор `explicit CanvasMemento(std::vector<std::unique_ptr<IShape>> shapes)` — принимает готовый вектор клонов.
- `GetShapesCopy() const` — возвращает **новую** копию всех фигур (через `Clone()`), чтобы Originator мог восстановиться, не портя сам снимок.

`include/memento/HistoryCaretaker.h` — новый файл

Что добавлено:

- Класс `HistoryCaretaker` — Caretaker.
- Два стека: `m_undoStack`, `m_redoStack` — `std::vector<std::unique_ptr<CanvasMemento>>`.
- `Save(std::unique_ptr<CanvasMemento>)` — кладёт снимок в `undoStack`, очищает `redoStack` (новое действие стирает будущее).
- `Undo(std::unique_ptr<CanvasMemento> currentState)` — берёт «до» из `undoStack`, текущее («после») кладёт в `redoStack`, возвращает «до».
- `Redo(std::unique_ptr<CanvasMemento> currentState)` — берёт «после» из `redoStack`, текущее («до») кладёт в `undoStack`, возвращает «после».

Ключевое отличие от старой версии: `Undo` / `Redo` принимают `currentState` — без этого `Redo` не работал (восстанавливал тот же снимок «до»).

`include/core/IShape.h`

Что добавлено:

```
virtual std::unique_ptr<IShape> Clone() const = 0;
```

Чисто виртуальный метод — база для снимка. Каждая фигура должна уметь возвращать свою точную копию.

`include/adapter/ShapeAdapter.h`

Что добавлено:

```
std::unique_ptr<IShape> Clone() const override
{
    return CloneImpl();
}

protected:
    virtual std::unique_ptr<IShape> CloneImpl() const = 0;
```

`Clone()` реализован в шаблоне и делегирует `CloneImpl()`, который реализуют конкретные фигуры.

`include/shapes/CTriangle.h` / `CRectangle.h` / `CCircle.h`

Что добавлено:

```
protected:
    std::unique_ptr<IShape> CloneImpl() const override;
```

Что убрано: ничего.

`src/shapes/CTriangle.cpp` / `CRectangle.cpp` / `CCircle.cpp`

Что добавлено:

```
std::unique_ptr<IShape> CTriangle::CloneImpl() const
{
    return std::make_unique<CTriangle>(*this);
}
```

include/shapes/CompositeShape.h

Что добавлено:

```
std::unique_ptr<IShape> Clone() const override;
```

src/shapes/CompositeShape.cpp

Что добавлено:

```
std::unique_ptr<IShape> CompositeShape::Clone() const
{
    auto copy = std::make_unique<CompositeShape>();
    for (const auto& child : m_children)
    {
        copy->Add(child->Clone());
    }
    return copy;
}
```

Рекурсивно клонирует всех детей — чтобы снимок группы был полным.

include/editor/Editor.h

Что добавлено:

```
void AddShape(std::unique_ptr<IShape> shape);
std::vector<std::unique_ptr<IShape>> GetShapesCopy() const;
```

```
void SetShapes(std::vector<std::unique_ptr<IShape>> newShape
s);
```

src/editor/Editor.cpp

Что добавлено:

```
void Editor::AddShape(std::unique_ptr<IShape> shape)
{
    m_shapes.push_back(std::move(shape));
}

std::vector<std::unique_ptr<IShape>> Editor::GetShapesCopy()
const
{
    std::vector<std::unique_ptr<IShape>> copy;
    copy.reserve(m_shapes.size());
    for (const auto& shape : m_shapes)
    {
        copy.push_back(shape->Clone());
    }
    return copy;
}

void Editor::SetShapes(std::vector<std::unique_ptr<IShape>> n
ewShapes)
{
    m_shapes = std::move(newShapes);
    m_selected.clear();
}
```

include/toolbar/ToolState.h

Что убрано:

- `#include "commands/CommandHistory.h"`
- поле `CommandHistory* m_history`
- `sf::Vector2f m_dragStartPos`
- `SetHistory(...)`

Что добавлено:

```
using CreateSnapshot = std::function<std::unique_ptr<CanvasMe
mento>()>;
using SaveSnapshot = std::function<void(std::unique_ptr<Canva
sMemento>)>;

void SetSnapshotCallbacks(CreateSnapshot create, SaveSnapshot
save);

void ResetDragState()
{
    m_isDragging = false;
    m_dragMoved = false;
    m_pendingSnapshot.reset();
}
```

Новые поля:

```
CreateSnapshot m_createSnapshot;
SaveSnapshot m_saveSnapshot;
std::unique_ptr<CanvasMemento> m_pendingSnapshot;
bool m_dragMoved = false;
```

Изменены сигнатуры фабрик:

```
static std::unique_ptr<ToolState> CreateDrag();      // было
unique_ptr<ITool>
static std::unique_ptr<ToolState> CreateFill();
```

```
static std::unique_ptr<ToolState> CreateAddShape(std::string
name, Creator creator);
```

Причина: `Toolbar` должен вызывать `SetSnapshotCallbacks` / `ResetDragState`, которых нет в `ITool`.

`src/toolbar/ToolState.cpp`

Что убрано:

- `#include "commands/AddShapeCommand.h"`, `ModifyShapesCommand.h`, `MoveShapeCommand.h`
- создание команд (`std::make_unique<AddShapeCommand>`, `MoveShapeCommand`) и `m_history->Push(...)`
- `m_dragStartPos`

Что добавлено / изменено:

Добавление фигуры:

```
if (m_creator)
{
    auto snapshot = m_createSnapshot ? m_createSnapshot() : n
ullptr;
    auto shape = m_creator(pos);
    if (shape)
    {
        if (snapshot && m_saveSnapshot) m_saveSnapshot(std::m
ove(snapshot));
        editor.AddShape(std::move(shape));
    }
    return true;
}
```

Снимок создаётся **до** `AddShape`, но кладётся в историю только если фигура реально создана.

Драг:

```
if (m_isDragMode && ok)
{
    m_isDragging = true;
    m_dragMoved = false;
    m_lastMousePos = pos;
    m_pendingSnapshot = m_createSnapshot ? m_createSnapshot()
: nullptr;
}
```

Снимок создаётся, но в историю **не** кладётся — ждём `MouseReleased`.

`MouseMoved`:

```
if (dx != 0.0f || dy != 0.0f)
{
    editor.MoveSelected(dx, dy);
    m_lastMousePos = pos;
    m_dragMoved = true;
}
```

`MouseReleased`:

```
if (m_isDragging)
{
    m_isDragging = false;
    if (m_dragMoved && m_pendingSnapshot && m_saveSnapshot)
    {
        m_saveSnapshot(std::move(m_pendingSnapshot));
    }
    m_pendingSnapshot.reset();
    m_dragMoved = false;
}
```

Если движения не было — снимок выбрасывается, в историю не попадает.

Фабрики возвращают `std::unique_ptr<ToolState>`.

`include/toolbar/Toolbar.h`

Что убрано:

- `#include "commands/CommandHistory.h"`
- поле `CommandHistory m_history`
- поле `std::unique_ptr<ITool> m_tool`

Что добавлено:

- `#include "memento/HistoryCaretaker.h"`
- `#include "toolbar/ToolState.h"`
- `HistoryCaretaker m_history`
- `std::unique_ptr<ToolState> m_tool` (было `ITool`)
- метод `void SetTool(std::unique_ptr<ToolState> tool)`

`src/toolbar/Toolbar.cpp`

Что убрано:

- `#include "commands/AddShapeCommand.h"`, `ModifyShapesCommand.h`
- `void Toolbar::MakeSnapshot()` — снимок теперь делает `ToolState`
- создание `ModifyShapesCommand` в кнопках цвета/толщины
- `m_history.Push(...)`
- `m_editor.CreateMemento()` / `RestoreFromMemento(...)`

Что добавлено / изменено:

`SetTool`:

```
void Toolbar::SetTool(std::unique_ptr<ToolState> tool)
{
    tool->SetSnapshotCallbacks(
        [this]() {
            return std::make_unique<CanvasMemento>(m_editor.G
etShapesCopy());
```

```
        },
        [this](std::unique_ptr<CanvasMemento> memento) {
            m_history.Save(std::move(memento));
        }
    );
    m_tool = std::move(tool);
}
```

Привязывает колбэки: «создать снимок» и «сохранить снимок».

Конструктор: `SetTool(ToolState::CreateDrag());` вместо `m_tool = ToolState::CreateDrag();`.

Кнопки инструментов: `SetTool(ToolState::CreateDrag());` и т.д.

Кнопки цвета/толщины:

```
m_history.Save(std::make_unique<CanvasMemento>(m_editor.GetShapesCopy()));
SetOutlineColorVisitor visitor(c);
for (IShape* s : sel) s->Accept(visitor);
```

Снимок делается **до** `Accept`.

`HandleEvent`: убран блок с `MakeSnapshot()` на клик по холсту. Осталась только обработка кнопок тулбара и передача события в `m_tool`.

`Undo` / `Redo`:

```
void Toolbar::Undo()
{
    auto current = std::make_unique<CanvasMemento>(m_editor.GetShapesCopy());
    auto memento = m_history.Undo(std::move(current));
    if (memento)
    {
        m_editor.SetShapes(memento->GetShapesCopy());
        if (m_tool) m_tool->ResetDragState();
    }
```

```
}

void Toolbar::Redo()
{
    auto current = std::make_unique<CanvasMemento>(m_editor.G
etShapesCopy());
    auto memento = m_history.Redo(std::move(current));
    if (memento)
    {
        m_editor.SetShapes(memento->GetShapesCopy());
        if (m_tool) m_tool->ResetDragState();
    }
}
```

Итоговая сводка

Файл	Убрано	Добавлено
`CanvasMemento.h`	—	весь файл
`HistoryCaretaker.h`	—	весь файл
`IShape.h`	—	`Clone()`
`ShapeAdapter.h`	`#include ShapeMemento`	`Clone()` / `CloneImpl()`
`CTriangle/CRectangle/CCircle.h`	—	`CloneImpl()`
`CTriangle/CRectangle/CCircle.cpp`	—	`CloneImpl()`
`CompositeShape.h`	—	`Clone()`
`CompositeShape.cpp`	`#include ShapeMemento`	`Clone()`
`Editor.h/.cpp`	—	`AddShape`, `GetShapesCopy`, `SetShapes`
`ITool.h`	`SetHistory`	—
`ToolState.h/.cpp`	`m_history`, команды, `m_dragStartPos`	колбэки снимка, `m_pendingSnapshot`, `m_dragMoved`, `ResetDragState()`

Файл	Убрано	Добавлено
`Toolbar.h/.cpp`	`CommandHistory`, команды, `MakeSnapshot`	`HistoryCaretaker`, `SetTool`, `Undo` / `Redo` с `currentState`